



SENSOR READINGS FROM A WALL-FOLLOWING ROBOT

DATA COLLECTED FROM A ROBOT WHILE
NAVIGATING AROUND A ROOM

COURSE: MACHINE LEARNING 2
STUDENT: MOBINA ALINAGHIAN
STUDENT NUMBER: S5905100

INTRODUCTION

- Dataset: <https://www.kaggle.com/datasets/uciml/wall-following-robot/data>
- The data were collected as the **SCITOS G5** navigated through the room following the wall in a **clockwise direction**, for **4 rounds**.
- To navigate, the robot uses **24 ultrasound sensors** arranged circularly around its "waist".



The original robot has 24 sensors, but the dataset **simplifies** these measurements into **four distances** representing the **front, left, right, and back** sides of the robot.

The dataset contains four input features:

- Front Distance
- Left Distance
- Right Distance
- Back Distance

These values represent the minimum measured distance in each direction around the robot.

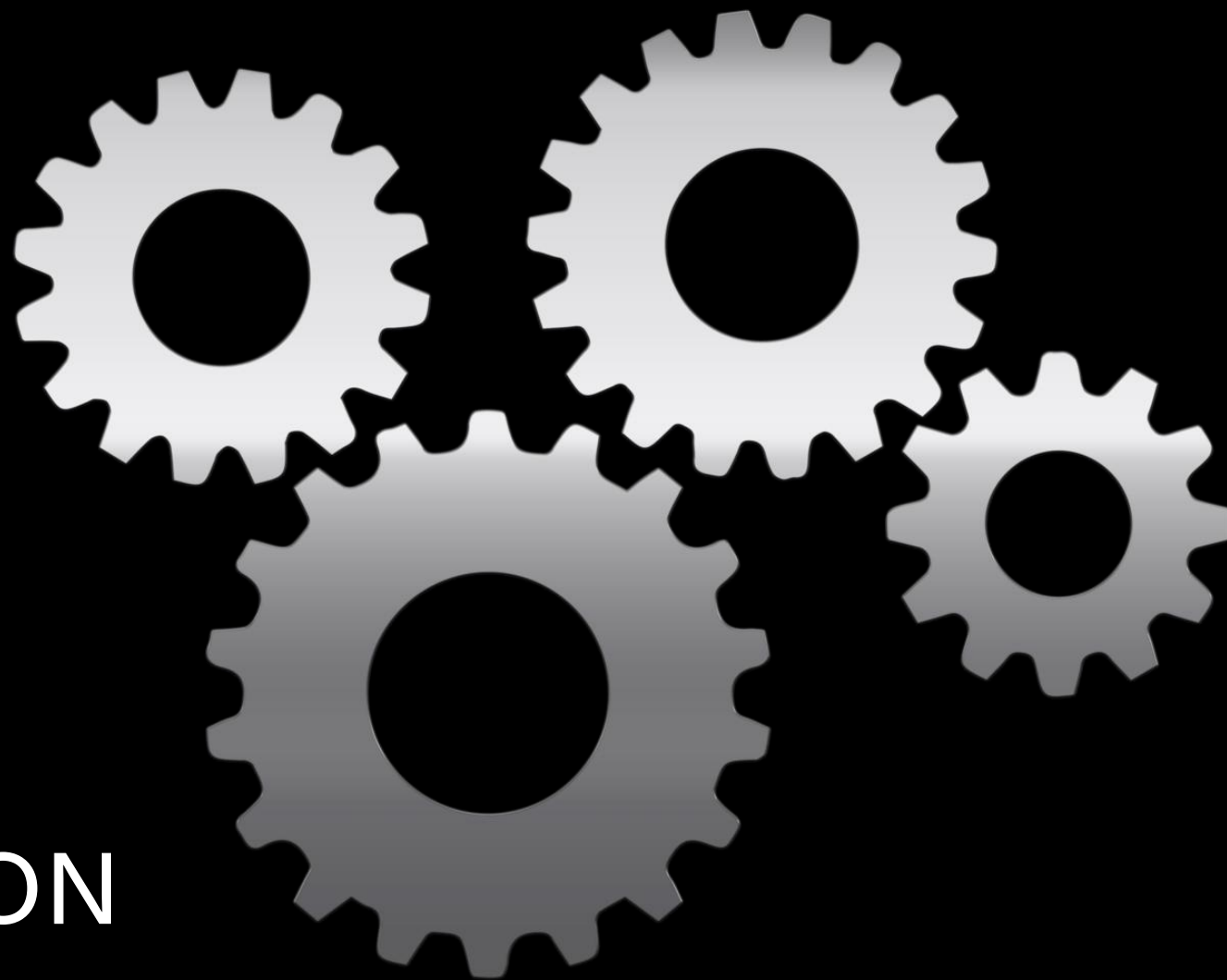
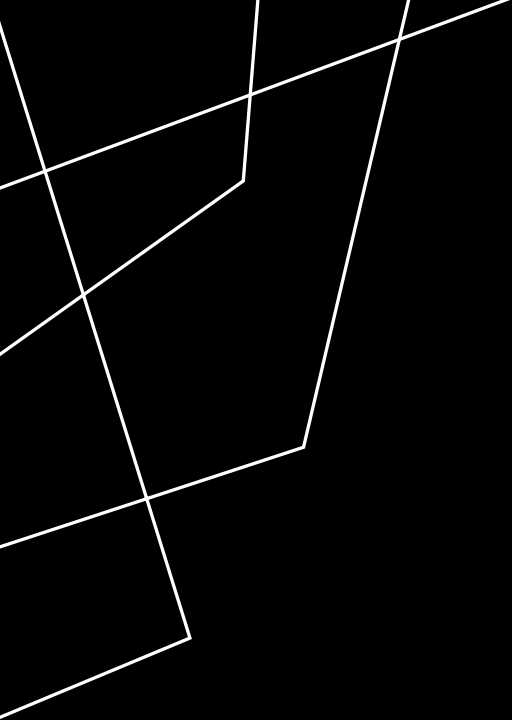


DATASET

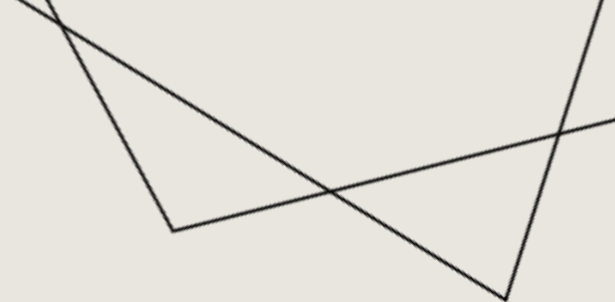


- The full structure of the dataset has 5456 instances.
- Each instance contains four sensor readings and one class label representing the robot's action.
- For each instance, the corresponding class label is given:
 - Move-Forward
 - Slight-Right-Turn
 - Sharp-Right-Turn
 - Slight-Left-Turn
- Arrangement: 5456 x 5 (4 sides + 1 class label)

index	SD_FRONT	SD_LEFT	SD_RIGHT	SD_BACK	Classes
0	1.687	0.445	2.332	0.429	Slight-Right-Turn
1	1.687	0.449	2.332	0.429	Slight-Right-Turn
2	1.687	0.449	2.334	0.429	Slight-Right-Turn
3	1.687	0.449	2.334	0.429	Slight-Right-Turn
4	1.687	0.449	2.334	0.429	Slight-Right-Turn
5451	1.024	0.657	1.087	1.562	Move-Forward
5452	0.894	0.649	1.071	1.085	Sharp-Right-Turn
5453	0.873	0.642	1.053	1.105	Sharp-Right-Turn
5454	0.967	0.635	1.034	1.118	Move-Forward
5455	0.854	0.628	1.016	1.168	Sharp-Right-Turn



IMPLEMENTATION



I am implementing two distinct machine learning algorithms, each representing different aspects of non-linear modeling:

- Support Vector Classifier (SVC)
- Random Forest

I trained the models using default fixed parameters. And then with a grid search process to optimize the hyperparameters of the algorithm.

Finally, I evaluated the models using accuracy and confusion matrices.

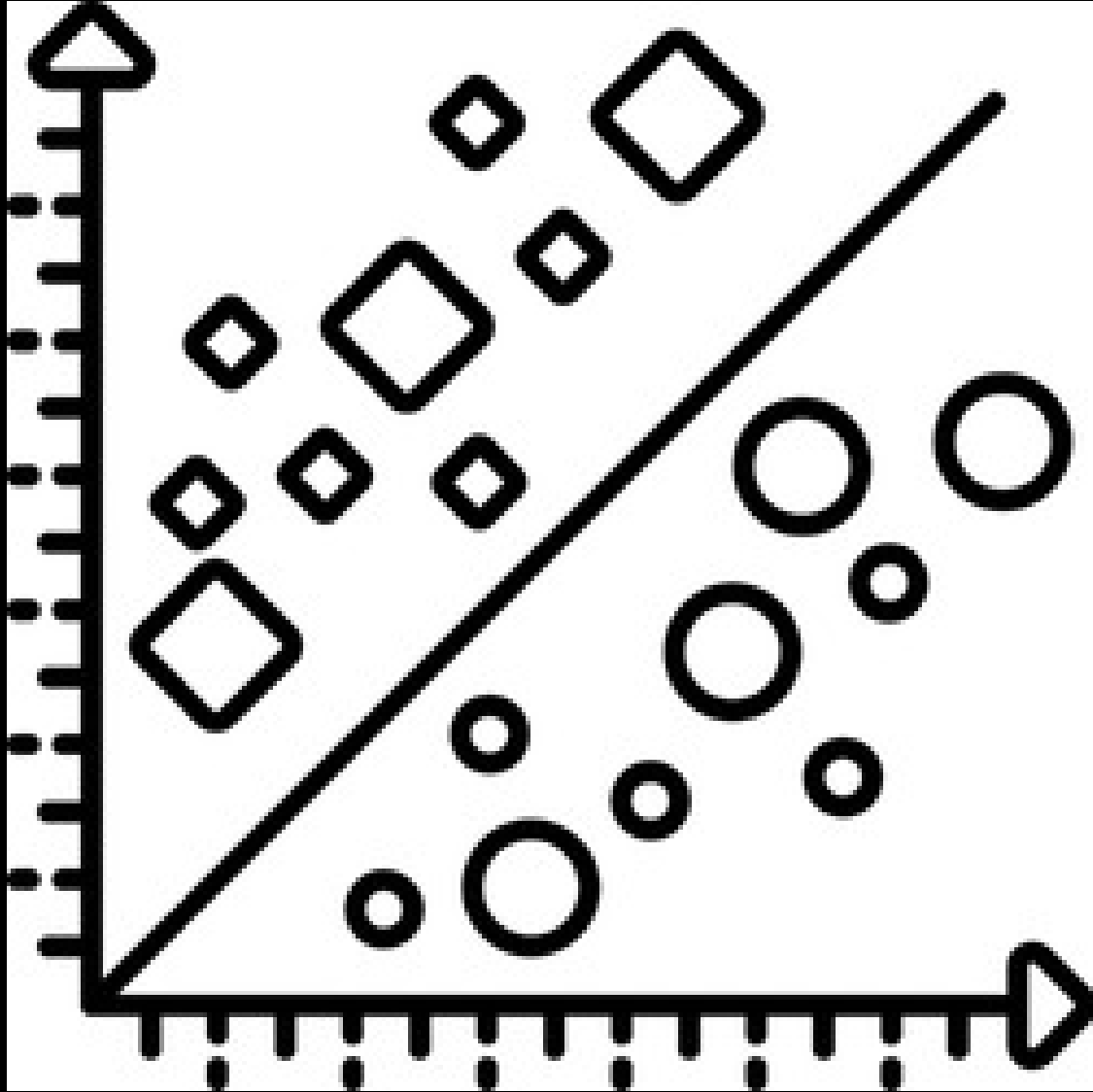
PRE-PROCESSING

The dataset is imported from a .csv file and loaded into the environment.

Data is then split into 3 subsets:

- the training phase (70%)
- the test phase (15%)
- the validation phase (15%)

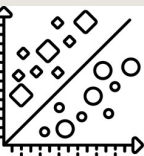
Apply the scaler to transform the training set and use the same scaler to transform the test and validation sets.

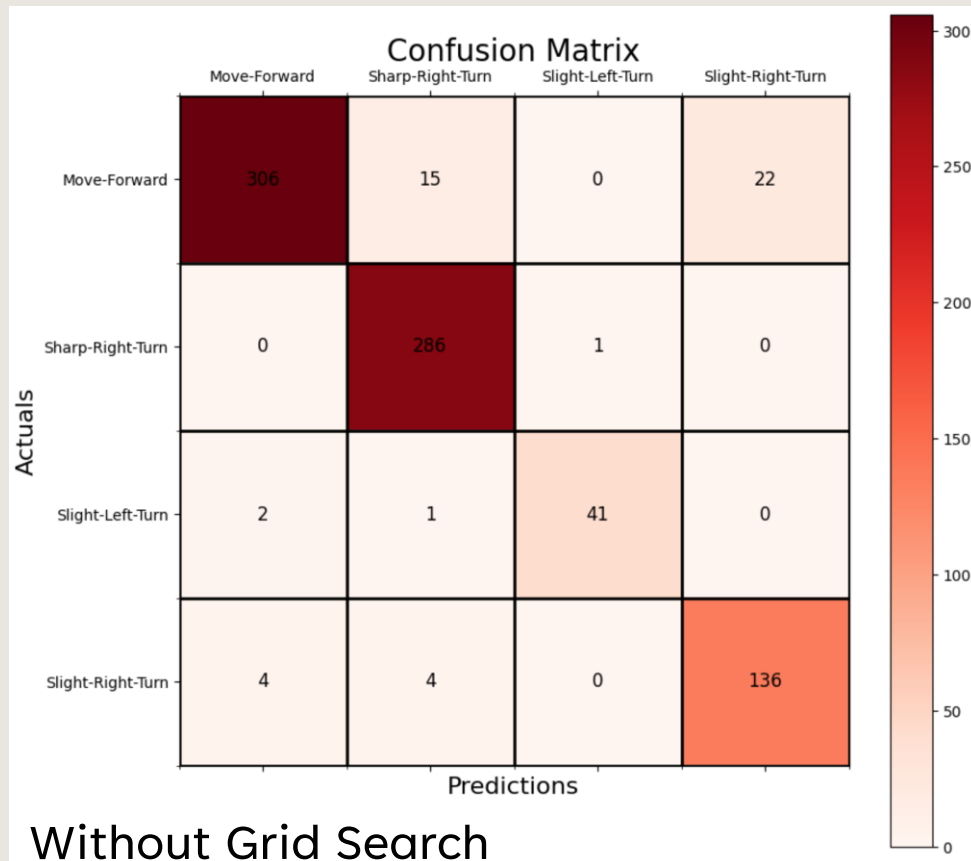


SVC

(SUPPORT VECTOR CLASSIFIER)

- 2D classification is projected into a higher dimension to find an optimal hyperplane to separate.
- The model's linearity is influenced by the gamma parameter, while the regularization parameter C controls the trade-off between margin width and misclassification rate.
- With the grid search approach, it is possible to tune the two parameters (gamma and C).

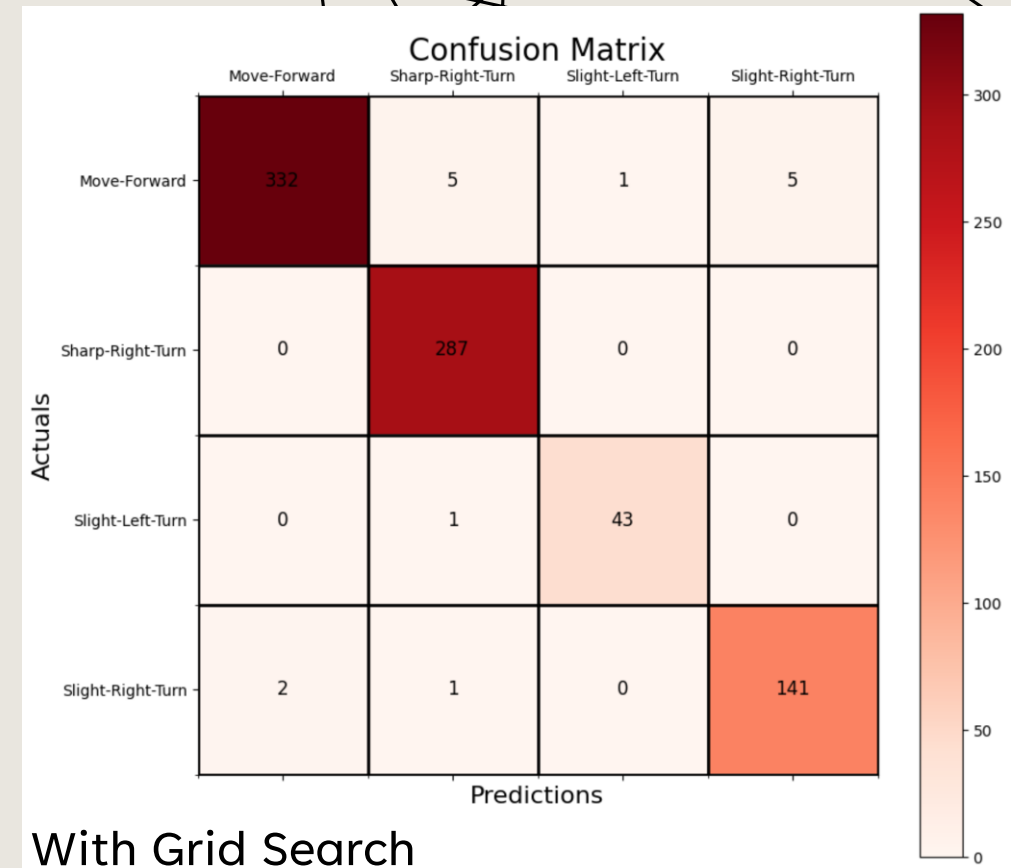




SVC accuracy on test data is: 94.01 %

Successful Navigation Rate: 94.01 %

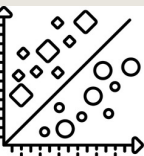
Collision Rate: 5.99 %

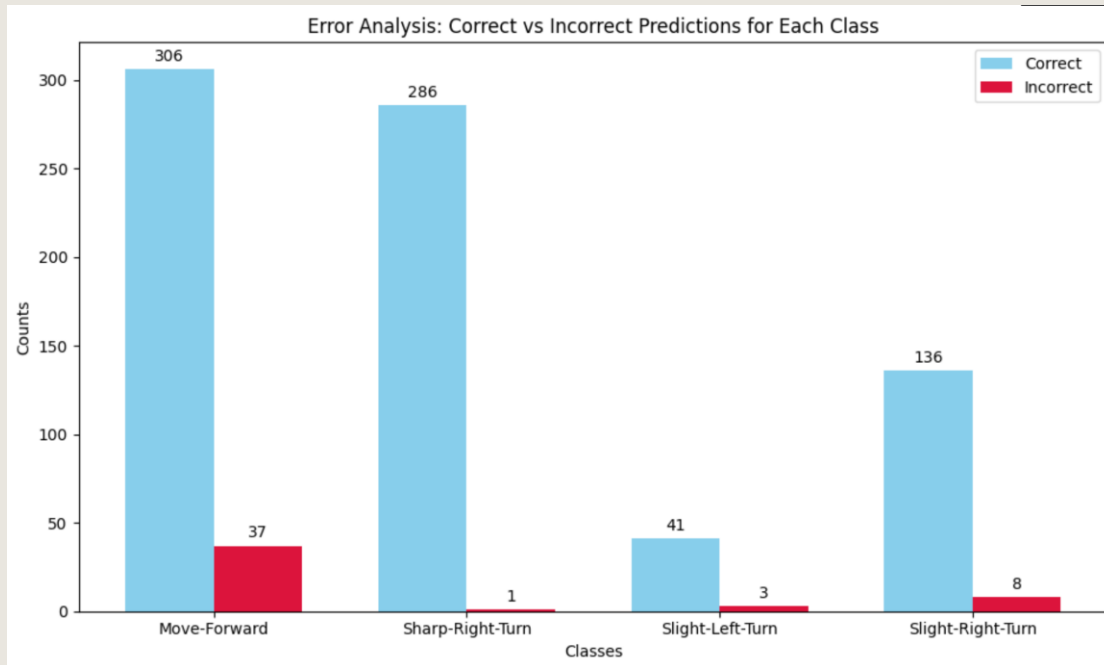
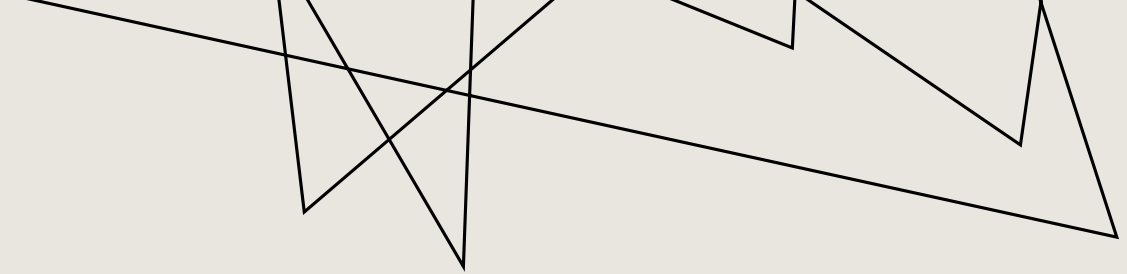


One-vs-Rest SVC accuracy on test data is: 98.11 %

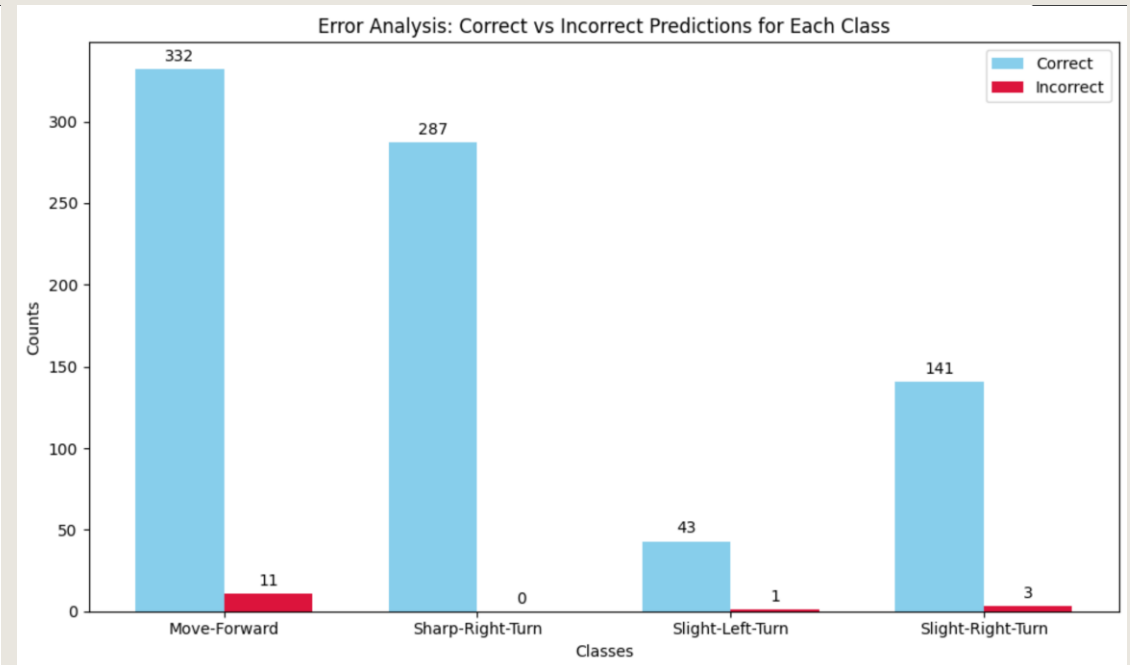
Successful Navigation Rate: 98.17 %

Collision Rate: 1.83 %

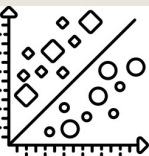


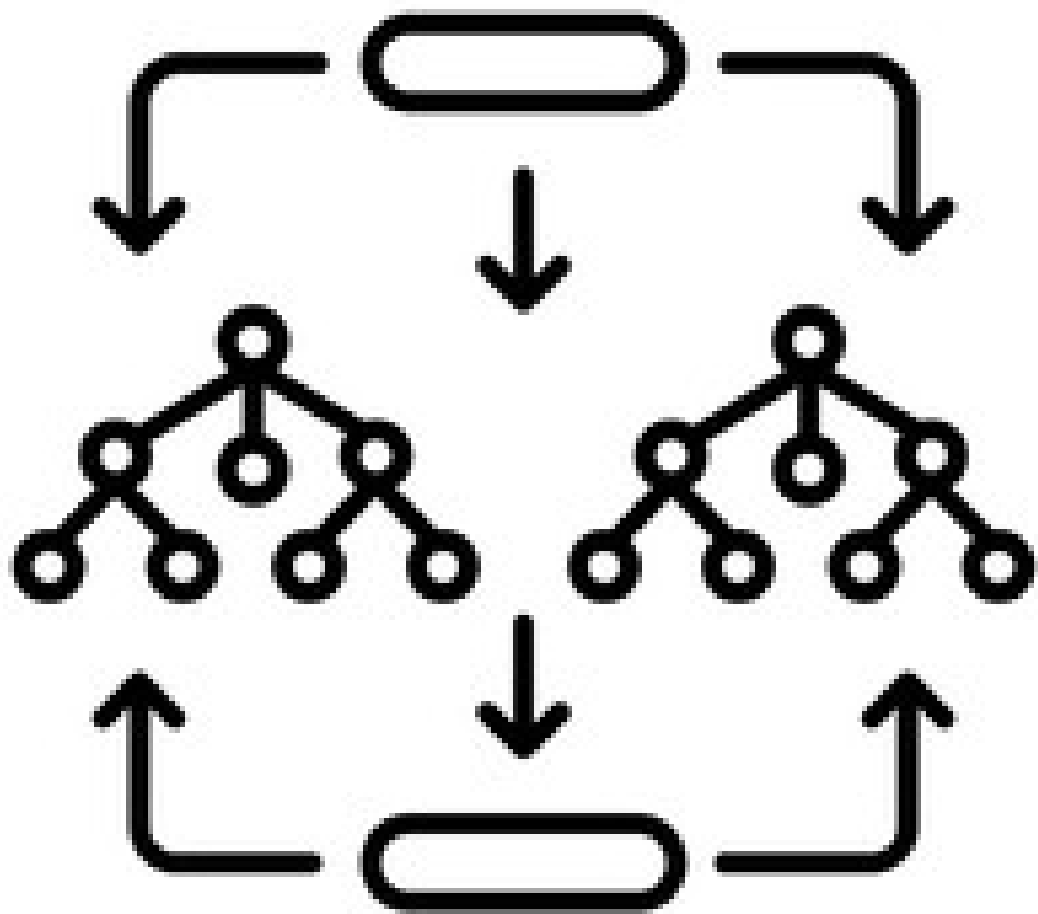


Without Grid Search

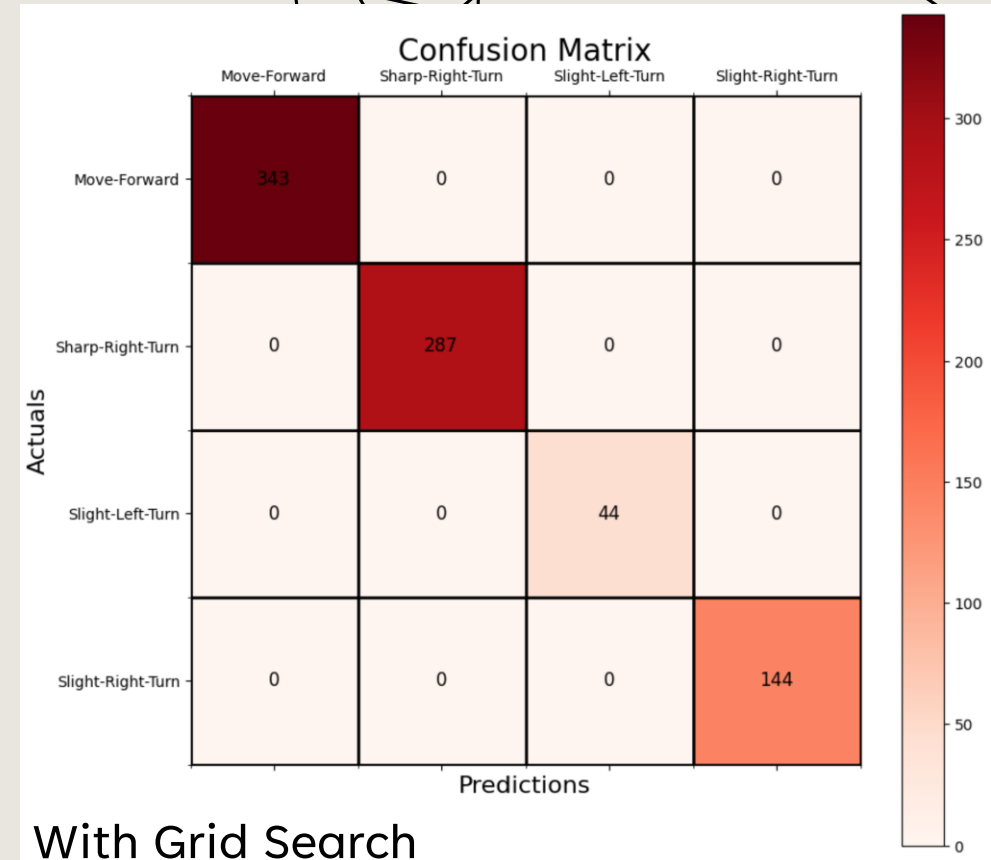
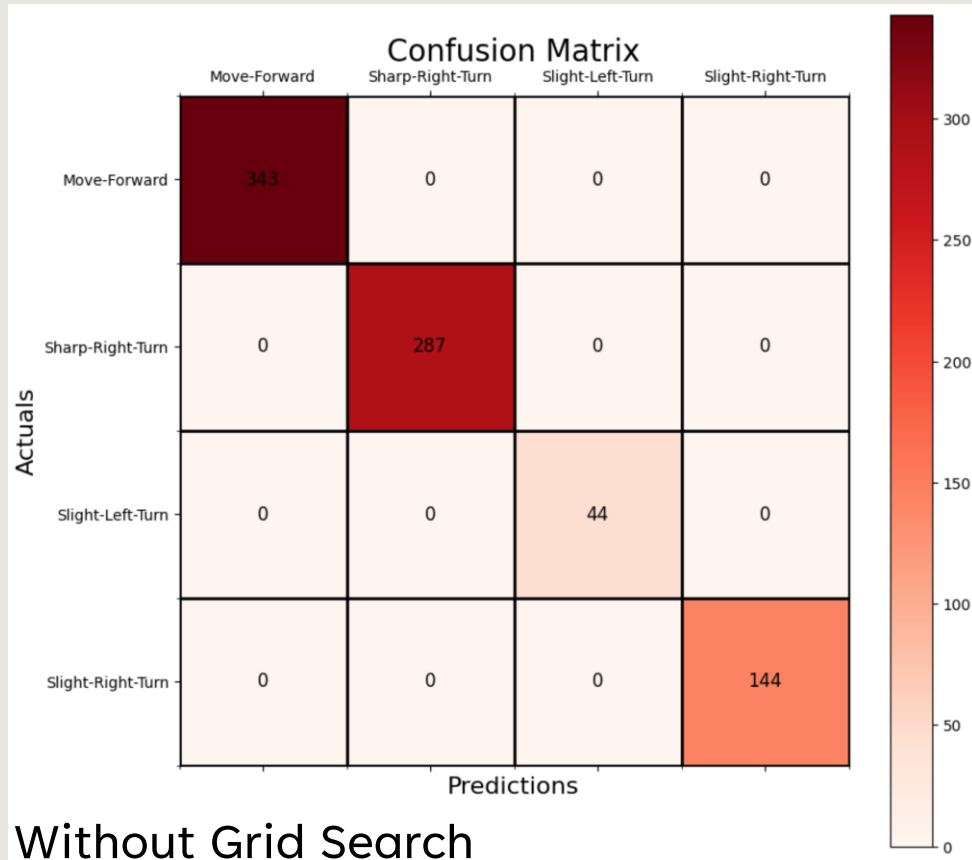


With Grid Search





RANDOM
FOREST



Random Forest accuracy on test data is: 100.00 %

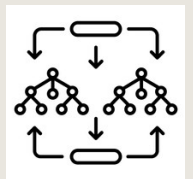
Successful Navigation Rate: 100.00 %

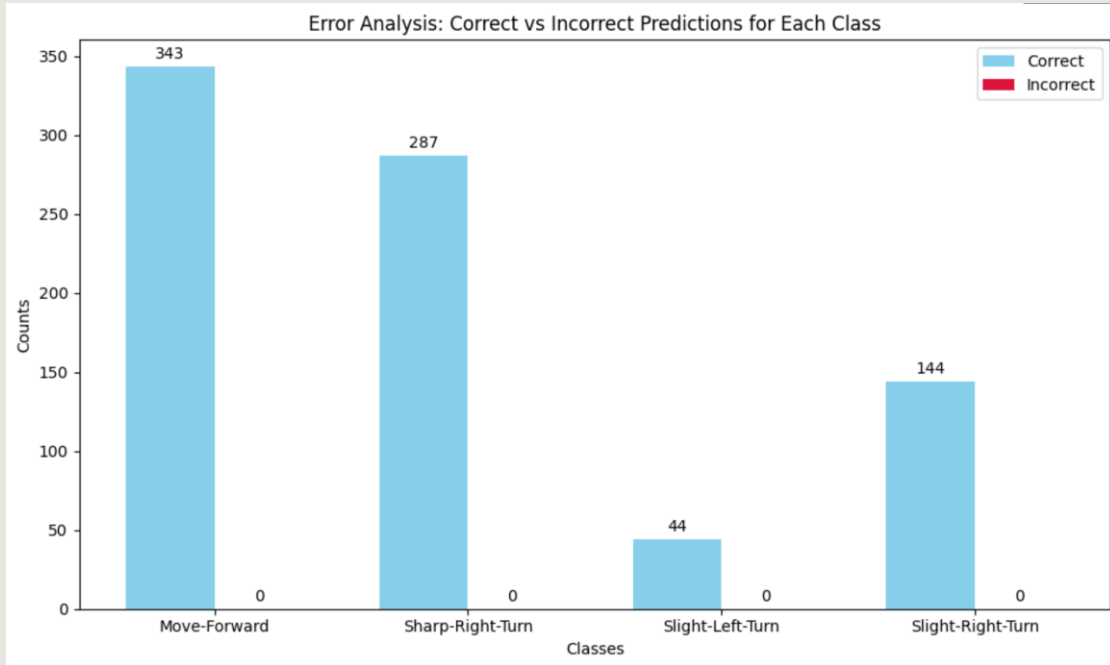
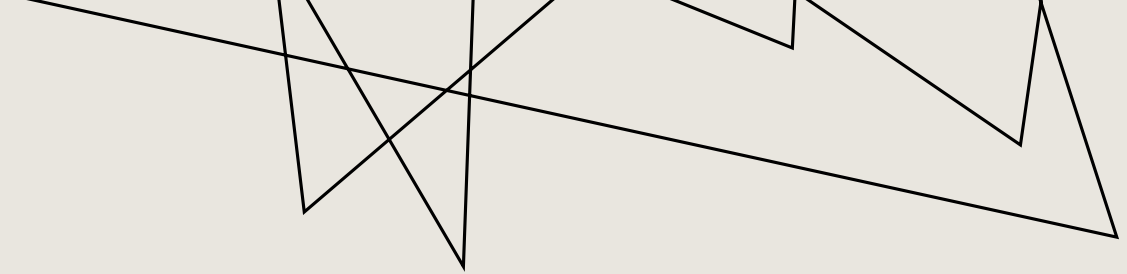
Collision Rate: 0.00 %

Random Forest accuracy on test data is: 100.00 %

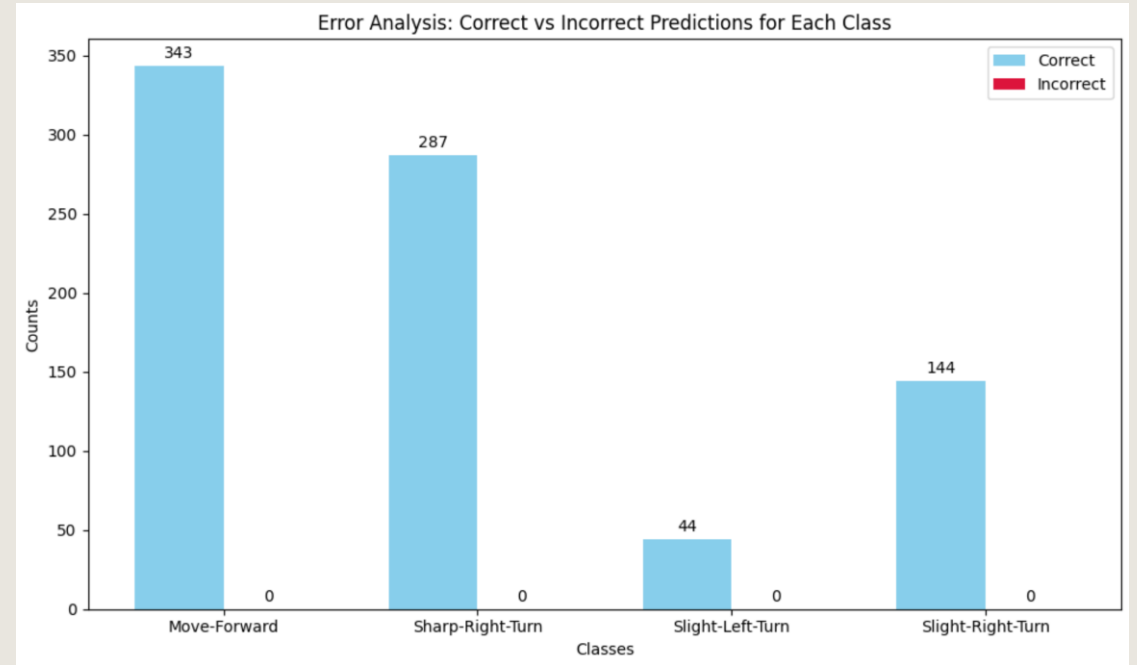
Successful Navigation Rate: 100.00 %

Collision Rate: 0.00 %

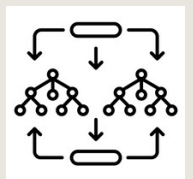




Without Grid Search



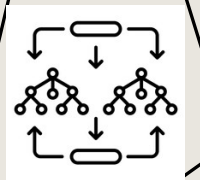
With Grid Search

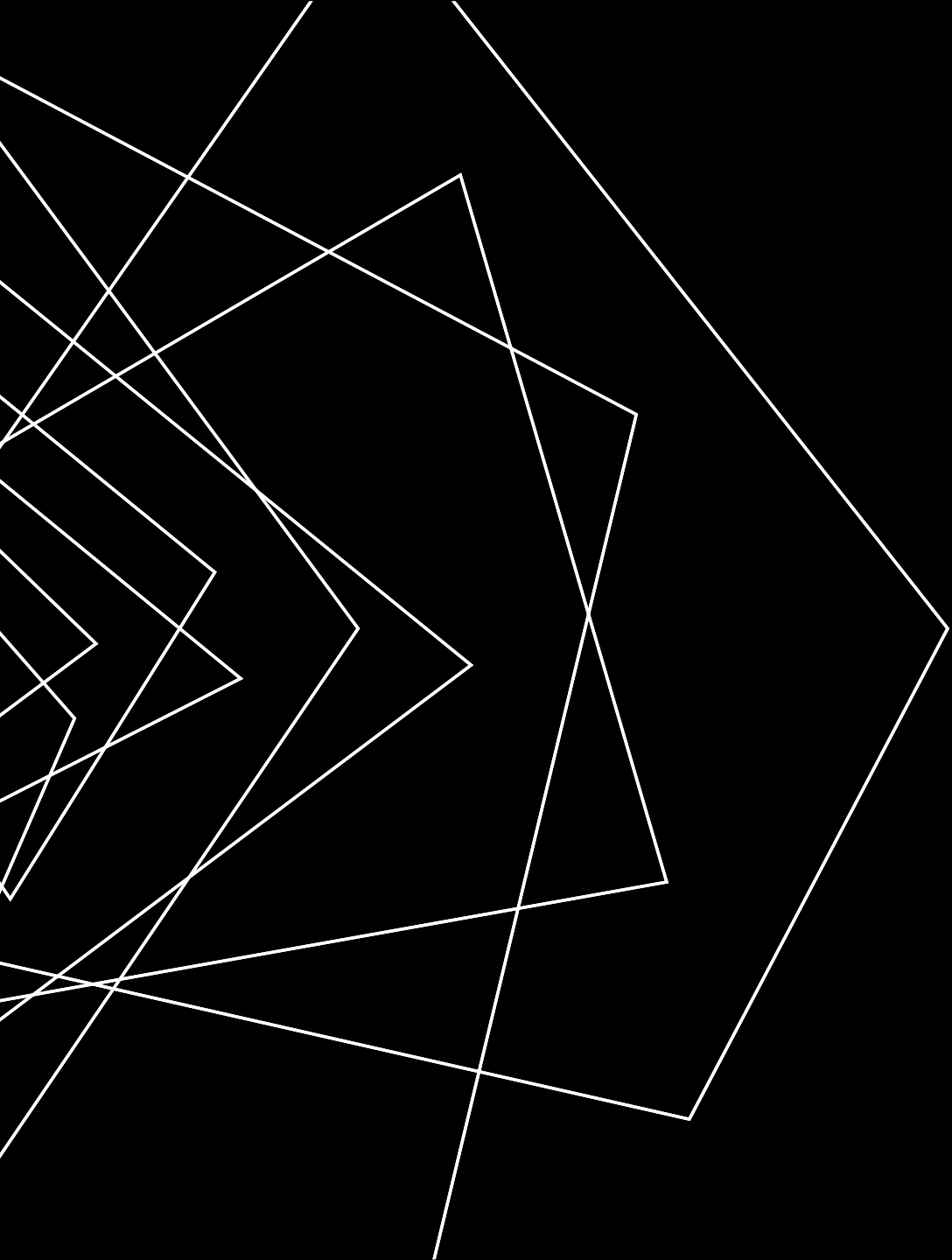


CONCLUSION

Based on the analysis of the results, it can be concluded that the most challenging classes to predict are “slight right turn” and “move forward,” as shown in the error analysis.

The Support Vector Classifier (SVC) performance is highly dependent on the choice of hyperparameters, which significantly influences its accuracy. Overall, the Random Forest algorithm demonstrated the highest performance between the evaluated models.





THANK YOU

Mobina Alinaghian